

MDFeed

실시간 마켓데이터 FEED 플랫폼

거래소 실시간 시세를 수집 → 정규화 → 멀티프로토콜 배포하는 금융 데이터 피드 서비스와,
그것을 리눅스에서 운영하기 위한 자동화 스택. 수집 경로 5개를 실연결로 검증했습니다.

저장소 github.com/oyeong011/market-feed-platform

데모 oyeong011.github.io/market-feed-platform

참조 데이터 github.com/oyeong011/financial-database

5	1,783	26	179
수집 경로 실연결 검증	코스피 종목 계층형 피드	실측으로 잡은 결함 기록	테스트 CI 6잡 통과

이 문서를 읽는 법

이 프로젝트에서 가장 값진 것은 기능 목록이 아니라 만드는 과정에서 잡아낸 결함 26건입니다. 문서가 틀린 곳, 측정이 거짓말하던 곳, 실제로 돌려봐야 드러나는 곳입니다. 각 항목마다 원인과 조치를 코드 주석과 문서에 남겼습니다.

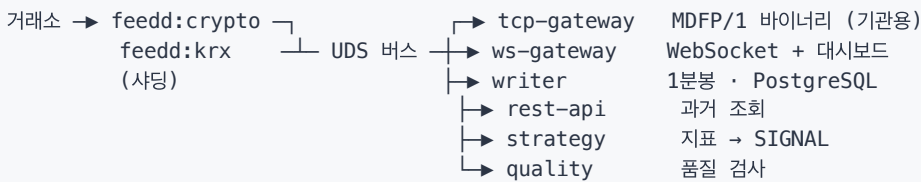
소유권 표기 — 이 프로젝트의 구현에는 AI를 상당 부분 활용했습니다. 이 문서에서 판단·측정·결정은 본인의 것이고, 코드 작성은 AI 보조입니다. 각 항목에서 그 경계를 구분해 적었습니다.

01 무엇을 만들었나

금융정보 서비스는 거래소에서 시세를 받아 고객 단말에 공급합니다. 그런데 거래소마다 형식이 전부 다릅니다.

업비트 JSON 을 바이너리 프레임으로 보냄
바이낸스 가격을 문자열로 보냄
한국투자 005930^103546^266250^... 파이프·캐럿 구분 문자열

이것을 하나로 통일해 여러 소비자에게 나눠주는 중간 계층이 **FEED 서비스**입니다. 매매 프로그램도, 시세를 보는 화면도 아닙니다. 그 화면들에 데이터를 먹여주는 뒷단입니다.



수집 경로와 자산군

자산군	경로	종목	갱신
암호화폐	업비트 · 바이낸스 WebSocket	7	실시간 틱
국내주식 (틱)	KIS WebSocket <code>H0STCNT0</code>	3	실시간 체결
국내주식 (전체)	KIS REST 계층형	1,783	15분 회전
지수	KIS 업종 현재지수	6	10초
해외금리	KIS 금리 종합	7	5분

무엇을 직접 만들고 무엇을 맡겼나

계층	선택	이유
피드 프로토콜·프레이밍·갭 복구	직접 구현	라이브러리를 써도 메시지 프레이밍·시퀀스 갭 탐지·스냅샷 복구·느린 구독자 격리는 어차피 직접 해야 합니다. 그 경계를 알려면 한 번은 아래까지 내려가 봐야 합니다.
분석 백테스트·성과지표	오픈소스	<code>backtesting.py</code> · <code>vectorbt</code> 가 낫습니다. 새로 만들 이유가 없습니다. 우리가 제공하는 건 데이터와 전략 코드이고, 검증은 검증된 도구에 맡깁니다.
저장소	오픈소스	PostgreSQL + TimescaleDB. 시계열 파티셔닝·보존정책을 직접 만들 이유가 없습니다.

피드 계층을 직접 만든 부수 효과로 핵심 경로 의존성이 0이 되어, `python3` 만 있으면 어떤 리눅스에서도 `git clone && make demo` 로
돌립니다.

02 프로젝트 STAR

S · 상황

채용 요건의 대부분이 이 계층에서 나옵니다 — 프로세스 간 통신, TCP/HTTP, 리눅스 서비스 점검, 파이프라인 자동화. **요건에 "안다"고 쓰는 대신 동작하는 것으로 보이기로 했습니다.**

T · 과제

기능을 늘리는 것이 아니라 **네 가지 실패를 막는 것**을 목표로 정의했습니다. 이 정의가 이후 모든 판단의 기준이 됐습니다.

문제	왜 어려운가
세션은 반드시 끊긴다	끊긴 구간은 영구 유실. 거래소는 과거를 다시 주지 않는다
느린 소비자 하나가 전체를 멈춘다	한 구독자를 기다리면 전원이 굼는다 (head-of-line blocking)
프로세스는 멀쩡한데 데이터만 안 온다	실제 장애의 대부분. <code>systemctl status</code> 로 안 잡힌다
틀린 값을 조용히 배포한다	값이 없는 것보다 나쁘다. 아무 알람도 안 울린다

A · 행동

① 구조 — 수집을 모으고 소비를 분리

소비자가 각자 거래소에 붙으면 같은 데이터를 N번 받고 N배로 끊깁니다. 수집은 한 프로세스로 모으고 배포·적재·전략·품질검사를 분리했습니다. 거래소 하나의 장애가 전체를 내리지 않도록 venue 그룹별로 사당했습니다.

② 느린 소비자 격리

발행 경로에서 전송 완료를 기다리면 구독자 하나가 느릴 때 전체가 멈춥니다. 구독자마다 유한 큐를 두고 차면 오래된 것부터 버리되, **버린 건수를 반드시 셉니다.**

③ 프로세스는 멀쩡한데 데이터가 안 오는 장애

`Restart=on-failure` 는 프로세스가 죽어야 동작합니다. liveness와 readiness를 분리하고, 헬스 판정 기반 위치독을 따로 뒀습니다.

④ 품질 검사 — 원칙을 집행하는 부분

원칙은 처음부터 적어 뒀는데 **집행 장치가 없었습니다.** 원칙만 있고 집행이 없으면 원칙이 아닙니다. 별도 프로세스가 6가지를 계속 검사합니다 — 가격 점프, 크로스된 호가, 광폭 스프레드, 정체, OHLC 위반, 교차 시장 괴리.

⑤ 그리고 계속 측정했습니다

참조 구독 클라이언트, 부하 시험, 장시간 감시, 장애 주입, 알람 검증. **측정하지 않았으면 몰랐을 것들이 계속 나왔습니다.**

R · 결과

측	결과
커버리지	수집 경로 5개 · 자산군 4축 · 코스피 1,783종목
무결성	구독자 데이터 무결성 48.5% → 100.0000%
성능	IPC 지연 p50 47.3 μ s · 파이프라인 49,235 msg/s · 적재 480,586 rows/s
안정성	systemd 11항목 자동 검증 · 장애 주입 4종 복구 확인
품질	108,156건 검사 → CRITICAL 0 · 암묵 환율 자산 간 괴리 0.050%
자동화	CI 6잡 · 테스트 179개 · 대시보드 지표 자동 갱신
기록	실측으로 잡은 결함 26건, 원인·조치 전부 문서화

I · 배운 것

① 문서를 읽기 전에 한 번 재본다

KIS 문서는 체결구분 3 이 매도라고 했지만 실제로는 5였습니다. 문서대로 봤으면 매도 체결이 전량 미상으로 남아 주문 흐름 지표가 통째로 무의미해졌을 것입니다.

② 근거 없이 적은 한계 서술은 없느니만 못하다

"구독자 수백 명이면 재인코딩이 병목"이라고 적어 뒀는데, 재보니 CPU 0.28%였습니다. 그럴듯한 문장은 아무도 의심하지 않고, 그 위에 잘못된 최적화 계획이 쌓입니다.

③ 안 돌아본 복구 경로는 복구 경로가 아니다

61분을 돌려도 재접속이 0회였습니다. 복구 코드가 맞는지 확인된 적이 없었다는 뜻입니다.

④ 계측 도구도 거짓말을 한다

알람 11개 중 4개가 영원히 안 울릴 상태였습니다. 누수 알람은 기동 때마다 오탐이 났습니다. 장애 주입 테스트 하나는 아예 안 돌고 통과했습니다.

⑤ 값이 없는 것보다 틀린 값이 나쁘다

국내 금리 응답이 손상돼 왔을 때, 그럴듯한 숫자를 내보내는 대신 발행하지 않고 그 이유를 노출하기로 했습니다.

03 실측으로 잡아낸 결함

전부 참조 클라이언트·통합 테스트·CI·장애 주입이 잡아낸 것입니다. 아래는 26건 중 대표 8건입니다.

증상	진짜 원인	조치
구독자 데이터 무결성 48.5%	게이트웨이가 심볼을 걸러 보내는데 시퀀스는 전역 번호를 그대로 흘림. 걸러진 번호가 구독자에게 전부 유실로 보였다. 프로토콜 설계 결함	구독자별 시퀀스 재넘버링 → 100.0000%
국내 매도 체결이 전부 방향 미상	KIS 문서는 체결구분 1 매수 / 3 매도라 했으나, 실제와 300건을 체결가와 호가로 대조하니 매도는 5.3은 한 건도 안 옴	관측값 기준 매핑. 미매핑 코드는 세어서 노출
feedd 재시작이 소비자 전체를 끌고 내려감	systemd Requires= 가 재시작을 전파. 버스 구독자에 넣어진 지수 백오프 자동 재접속이 설정 한 줄로 무력화	Wants= + After= 실측: feedd만 111→170
종료 중 세그폴트	asyncio.to_thread 는 await를 취소해도 스레드가 안 멈춤. 그 스레드가 executemany 중일 때 DB 커넥션을 닫아 use-after-free	DB 접근 경로를 단일 락으로 직렬화
지연시간이 -30,825 μs	거래소 시계가 로컬보다 앞섬. 음수 지연이 든 테이블은 어떤 SQL 집계를 해도 결론이 틀림	NTP 방식 최소값 필터로 venue별 오프셋 보정
벤치마크가 지연이 아니라 큐 적체를 측정	최대 속도로 밀어 넣고 지연을 재니 p50이 105 ms. IPC 지연이 아니라 기다린 시간	지연은 페이싱, 처리량은 버스트로 분리 → 47 μs
알람 11개 중 4개가 영원히 안 울릴 상태	카운터를 0으로 초기화하지 않아 첫 사건 전까지 지표가 없음. Prometheus는 오류 없이 조용히 no data	사전 초기화 + CI에서 지표 실재 검증
이동평균 동물을 교차로 오인	a > b 하나로 판정해 두 값이 정확히 같아지는 순간을 "하향 돌파"로 처리. 국내 주식은 호가 단위가 커서 실제로 발생하는데 크립토 float에서는 거의 없어 오래 숨어 있었음	동물은 판정 보류. 수정 후 교차 9곳 pandas와 일치

이 목록이 이 프로젝트의 알맹이입니다

기능은 코드를 읽으면 알 수 있습니다. 하지만 "문서가 틀렸다"를 어떻게 알았는가, "내가 쓴 한계 서술이 틀렸다"를 어떻게 발견했는가는 과정을 남겨야만 전달됩니다.

04 채용 조건 대응

요건	대표 근거	실측
금융 데이터 FEED 개발 및 운영	수집 경로 5개, 배포 프로토콜 3종, 스냅샷+증분 복구	무결성 48.5% → 100.0000%
Linux 서비스·프로세스 점검 및 안정화	systemd 6유닛 + 자동 검증 테스트베드 + 장애 주입	11항목 · 복구 4종 확인
파이프라인·배포·점검 자동화	Makefile 단일 입구 · CI 6잡 · Pages 지표 자동 갱신	테스트 179개
Python	수집·배포 hot path 및 운영 도구 전부	소스 8,835줄 · 핵심 의존성 0
SQL 기초, 관계형 DB 이해	복합 인덱스 · 사전 집계 upsert · 하이퍼테이블 · 보존정책	적재 480,586 rows/s
Linux 명령·프로세스· 로그 확인	ops.sh diag 원스톱 진단 · RUNBOOK 장애 8종	1차 진단 한 줄
프로세스 간 통신, 네트워크 프로토콜	MDFP/1 TCP · WebSocket 클라이언트·서버 · HTTP/1.1 서버 · UDS pub/sub · 공유메모리 링버퍼 직접 구현	IPC 지연 p50 47.3 μs
자료구조·운영체제· 네트워크	링버퍼 · 로그버킷 히스토그램 · 시그널 · AIMD 유량 제어	세그폴트·종료정지 실사례
우대 · 파이프라인· 인프라·자동화	SEC EDGAR · OpenDART 수집기 143일 운영	487,434건 · error 0
우대 · Feed 기반 마켓데이터 서비스	KIS 실계좌 연결 · 계층형 피드 설계 · 품질 검사	코스피 1,783종목

네트워크·IPC — 직접 구현한 것

구현물	다른 문제
MDFP/1 TCP 프로토콜	TCP는 바이트 스트림이라 메시지 경계가 없다. 길이 프리픽스 프레이밍 · CRC32 무결성 · 시퀀스 갭 탐지 · 재동기화 · 스냅샷+증분
WebSocket 클라이언트·서버 RFC 6455	HTTP Upgrade 핸드셰이크 · Sec-WebSocket-Accept 계산 · 마스크 · 확장 길이 (7/16/64비트) · 단편화 재조립 · ping/pong
HTTP/1.1 서버	keep-alive · Content-Length 바이트 정확도 · HEAD · 헤더 크기 상한 · 경로 탈출 차단
UDS pub/sub 버스	프로세스 간 팬아웃 · 구독자별 백프레서 · 지수 백오프 재접속
공유메모리 링버퍼	SPSC 브로드캐스트 · 논블로킹 생산자 · 추월 감지 · 찢긴 읽기 방지

05 어디까지 검증했나

실측 성능

계층	처리량	지연 P50	지연 P99
MDFP 인코딩	2,305,570 msg/s	434 ns	—
MDFP 파싱 (재동기화 포함)	759,786 msg/s	1,316 ns	—
공유메모리 링버퍼	1,560,551 msg/s	641 ns	—
UDS 순수 IPC 지연	—	47.3 μs	105.9 μs
HTTP/1.1 (keep-alive)	15,237 req/s	62.2 μs	95.5 μs
전체 파이프라인	49,235 msg/s	—	—

macOS 26.5 · Apple Silicon 10코어 · Python 3.14. 절대 성능이 아니라 계층별 상대 비용과 이 구조가 견디는 규모를 보기 위한 수치입니다.

부하 — 어디서 무너지는가

리플레이를 3배속으로 고정해 소스 속도를 회차마다 동일하게 두고 했습니다. 실거래소는 틱 밀도가 회차마다 달라 구독자 수에 따른 변화인지 구분할 수 없습니다. 비교하려면 변수를 하나만 남겨야 합니다.

구독자	총 처리량	1인당 유지율	P99 지연	유실	드롭
1	40 msg/s	100%	1.4 ms	0	0
10	491	124%	3.7 ms	0	0
25	1,330	139%	8.0 ms	0	0
50	2,507	130%	12.8 ms	0	0
100	6,376	168%	27.9 ms	0	0

처리량은 버팁니다. 총 처리량이 160배 늘어도 유실·드롭이 0이고 1인당 처리량이 유지됩니다 — 팬아웃은 선형입니다. 무너지는 건 지연입니다(p99 19배). 이 배포단의 한계는 "몇 명까지"가 아니라 "몇 ms까지 허용하나"입니다.

이 측정이 제 가설을 뒤집었습니다

설계 문서에 "구독자 수백 명이면 재인코딩이 병목"이라고 적어 뒀습니다. 몇 명인지 재보지도 않고 쓴 문장이었습니다. 재보니 100명·6,376 msg/s에서 재인코딩 CPU는 0.28%였습니다. 단계를 쪼개니 구독자당 재인코딩 552 ns, 큐 삽입 410 ns, 소켓 write+drain 1,476 ns — 진짜 비용은 소켓 쓰기였고, 재인코딩을 없애도 4분의 1만 줄어듭니다.

장애 주입 — 복구 경로는 평소에 안 돌아간다

지수 백오프 재접속, CRC 재동기화, 느린 구독자 격리, DB 폴백, 샤드 격리. 그런데 61분을 돌려도 재접속이 0회였습니다. 안 돌아본 복구 경로는 복구 경로가 아닙니다.

시나리오	결과
TCP 스트림에 쓰레기 주입	오염 3회 → 재동기화 4회 후 1,101건 계속 수신
읽지 않는 구독자 25초	게이트웨이 정상 유지
DB 파일 권한 박탈	writer 생존 · 복구 후 적재 재개 (32,429 → 35,271)
<code>feedd</code> SIGKILL	writer PID 유지 + 수신 재개 (38,047 → 39,975)
샤드 하나 SIGKILL	다른 샤드 정상 · 적재 계속 (4,533 → 6,978행)

인프라 실검증

항목	확인한 것
PostgreSQL + TimescaleDB	하이퍼테이블 3개 · 보존정책 2개 · 분석 뷰 4종 · SQLite와 동시 적재로 다중 소비자 검증 (시퀀스 갭 0)
Docker Compose	7개 컨테이너 전부 healthy · REST/TCP/대시보드 전 구간 · 구독 무결성 100%
리눅스 systemd	Ubuntu 22.04 테스트베드에서 11항목 자동 검증 — <code>User=mdfeed</code> · <code>ProtectSystem=strict</code> · SIGKILL 재시작 · 우아한 종료 flush
Prometheus + Grafana	타킷 8개 UP · 알람 11개 4그룹 · 대시보드 12패널 자동 프로비저닝 · 알람 지표 실재 검증을 CI에 편입

데이터 품질 — 원칙을 집행하는 부분

"틀린 값을 조용히 배포한다"를 네 문제 중 하나로 적어 놓고, 정작 탐지 장치가 없었습니다. 별도 프로세스가 6가지를 계속 검사합니다.

항목	결과
검사한 메시지	108,156
CRITICAL	0
WARNING	92 (전부 실제 시장 움직임)

외부 소스 없이 자체 정합성을 검사합니다. 업비트는 원화, 바이낸스는 달러로 같은 코인을 거래합니다. 두 가격의 비율이 곧 환율이고, 여러 코인에서 뽑은 값은 서로 가까워야 합니다.

BTC 1,386.7	ETH 1,386.2	SOL 1,386.0	KRW/USD	괴리 0.050%
-------------	-------------	-------------	---------	-----------

세 종목이 서로를 검증합니다. 하나만 벌어지면 그 종목의 시세가 이상하다는 뜻이고, 어느 쪽인지는 나머지가 알려줍니다. **이상을 발견해도 데이터를 버리지 않습니다** — "이상해 보이는 값"의 상당수는 진짜 시장 움직임(급등락·유동성 고갈·서킷브레이커)입니다. 표시하고 기록할 뿐, 판단은 사람이 합니다.

06 범위와 한계

정직하게 적습니다. 못 하는 것을 한 척하지 않는 것이 이 프로젝트의 원칙입니다.

한계	내용
국내 주식 틱은 3종목까지	계정 한도이며 실측으로 확인했습니다. 나머지 1,780종목은 REST 스냅샷으로 덮었고 체결 단위가 아닙니다 . 코스피 전종목 틱은 거래소와의 정식 시세 계약 영역이고 개인 API로는 불가능합니다.
배포단의 한계는 구독자 수가 아니라 지연	100명까지 유실·드롭 0이지만 p99가 1.4 ms → 27.9 ms로 19배 늘어납니다.
국내 금리는 수집하되 발행하지 않음	공급 측 응답에서 종목명이 U+FFFFD 로 이미 손상돼 오고 카·값 대응도 어긋납니다. 짝짓기를 신뢰할 수 없어 발행하지 않고 그 사실과 손상 건수를 노출 합니다.
호가는 최우선(BBO)만	전체 depth 오더북 복원은 범위 밖입니다.
주문 실행 없음	시그널 생성까지입니다. 실주문은 전혀 다른 신뢰성 요구를 갖습니다.
시계 오프셋은 상대값	편도 지연의 절대값을 알려면 PTP 하드웨어 타임스탬프가 필요합니다.
백테스트 결과는 통계적 의미 없음	저장소 샘플은 38분 구간 단일 종목입니다. 파이프라인이 동작한다는 증거이지 전략의 우열이 아닙니다. 세 전략 모두 손실이었고 그대로 적었습니다.
PINGPONG 실측 미검증	코드는 공식 예제와 맞췄으나 데이터가 흐르는 동안엔 ping이 오지 않아 실제로 받아보지 못했습니다.

산출물

구분	내용
소스	8,835줄 — 프로토콜 · 어댑터 5종 · 서비스 7개 · 저장소 · 지표 · 품질
테스트	1,879줄 · 179개 — 프로토콜 퍼징 · 지표 대조 · E2E · 샤딩 · 품질
운영	1,232줄 — systemd 6유닛 · <code>ops.sh</code> · 워치독 · 헬스체크 · 장애 주입 · Prometheus/Grafana
문서	README · PORTFOLIO · DETAILS · DESIGN · RUNBOOK